

WEB DEVELOPMENT PRACTICE NOTES

A clean, beginner-friendly guide based on the code practiced from Day 1 to Day 16.

What you have practiced

HTML structure and forms • CSS styling and layout • Tailwind CSS • JavaScript fundamentals • Arrays and objects • Events and forms • React • Vite and project structure

How to use these notes

Each topic gives you a plain-language explanation followed by a small example. Keep HTML/CSS, JavaScript, and JSX in their appropriate files when practicing.

Core skill map

HTML	Structure, headings, sections, links, images, forms, labels, inputs, buttons, semantic elements
CSS	Selectors, colors, fonts, spacing, borders, box sizing, Flexbox, responsive media queries, variables, pseudo-elements
Tailwind CSS	Utility classes for layout, spacing, colors, breakpoints, and styling
JavaScript	Variables, functions, conditions, arrays, objects, map(), filter(), spread, destructuring, modules, form events
React	JSX, components, props, lists, conditional UI, events, useState, and Vite projects

1. HTML — Structure and Forms

HTML gives content meaning and structure. Tags create elements, while attributes add details.

Semantic elements

Semantic tags describe the purpose of a page region: **<header>**, **<nav>**, **<main>**, **<section>**, and **<footer>**.

```
<main><h1>Welcome</h1><p>Start here.</p></main>
```

Forms and inputs

A form collects user input. Connect a label to its input using matching **for** and **id** values.

```
<label for="email">Email</label><input id="email" type="email" required>
```

Common input types include **email**, **password**, **checkbox**, and **text**. The **required** attribute enables basic browser validation. Use a button with **type="submit"** to submit a form.

Links and images

A link navigates to another location, while an image displays a file. Meaningful images should have useful alternative text.

```
<a href="/about">About</a>\n
```

2. CSS — Style and Layout

CSS controls how HTML looks. A selector chooses elements, and declarations set their properties.

```
.card { color: #222; padding: 16px; border: 1px solid #ddd; }
```

Selectors and CSS variables

A class selector starts with a dot. A CSS variable stores a reusable value.

```
:root { --brand: purple; }\n.button { background: var(--brand); }
```

The box model

The box model consists of **content**, **padding**, **border**, and **margin**. This rule makes element sizing easier to reason about:

```
* { box-sizing: border-box; }
```

With **border-box**, an element's declared width includes its padding and border.

Flexbox and responsive design

Flexbox arranges children in a row or column. Use **gap** for consistent space between children.

```
.row { display: flex; align-items: center; justify-content: space-between; gap: 16px; }
```

Responsive design adapts the layout to different screen widths using media queries.

```
@media (max-width: 700px) {\n  .row { flex-direction: column; }\n}
```

Pseudo-classes and pseudo-elements

Pseudo-classes style a state, while pseudo-elements add decorative content.

```
a:hover { text-decoration: underline; }\n.box::before { content: ""; }
```


Spread syntax, destructuring, and modules

Spread syntax (...) copies an object's enumerable properties into a new object. Destructuring pulls named properties into variables. Import/export allows code to be shared across files.

```
const updated = { ...person, age: 21 };
const { name } = person;
export const siteName = "StudyHub";
import { siteName } from "../content.js";
```

6. Events and Form Handling

An event listener runs a function when something happens. **preventDefault()** stops a form's usual page navigation so JavaScript can handle the submission. **checkValidity()** checks required fields and input types in the browser.

```
form.addEventListener("submit", (event) => {\n event.preventDefault();\n});
```

After preventing the default submission, read the values and show feedback as appropriate. Browser-side validation is useful for learning, but a real login also needs a secure server.

7. React — Components and JSX

A component is a JavaScript function that returns JSX (HTML-like UI). Reuse it by writing its component name as a JSX tag. Component names begin with a capital letter.

```
function Greeting() {\n return <h1>Hello!</h1>;\n}
```

JSX basics

JSX uses **className** instead of HTML **class**. JavaScript values go inside curly braces.

```
const title = "Welcome";\nreturn <h1>{title}</h1>;
```

Props

Props are inputs passed from a parent component. They let a component display different data.

```
function Badge({ text }) {\n return <span>{text}</span>;\n}\n\n<Badge text="New" />
```

8. React — Lists, Events, and State

Lists

Render repeated UI by mapping data to components. Give each item a stable **key** so React can track it when the list changes.

```
pets.map((pet) => (\n <PetCard key={pet.name} name={pet.name} />\n))
```

Conditional rendering

Conditional rendering shows UI only when a condition is true.

```
{showMenu ? <nav>Menu</nav> : null}
```

Event handlers

An event handler responds to a click or another event.

```
<button onClick={() => setOpen(true)}>Open</button>
```

State with useState

State is a component's changeable memory. **useState** gives the current value and a setter. Call the setter to request an update; do not change state directly.

```
const [count, setCount] = useState(0);\nsetCount((old) => old + 1);
```

9. Vite and Project Structure

Vite is the development and build tool used in your React folders.

Typical project roles

index.html provides the root element. **main.jsx** mounts React there. **App.jsx** and other components describe the page. **package.json** records dependencies and commands.

```
createRoot(document.getElementById("root")).render(<App />);
```

Useful commands

npm run dev starts the development server. **npm run build** creates production files.

10. What to Review Next in Your Own Code

Some JavaScript examples are commented out, so they are practice notes rather than code that currently runs. Day 16 has placeholder components that return nothing. Try completing them and rendering the cards from your existing **cardsContent** array.

Review checklist

- Check React list items for a **key** prop.
- In JSX SVG, use **className**, **strokeWidth**, **strokeLinecap**, and **strokeLinejoin**.
- In HTML, avoid nesting one **form** inside another **form**.

Next practice project

Create a list of books, display each with a reusable Card component, filter the list by category, and add a button that toggles a details panel.

Quick Formula Recap

map()	Each item becomes a new item.
filter()	Keep items when a condition is true.
function	Inputs → instructions → return value.
Flexbox	display: flex + alignment + gap.
React state	Current value + setter → UI updates.
CSS responsive rule	Breakpoint → style.

Final note

These notes describe topics visible in the practice files. They are not a score or a claim that every topic is mastered. Keep experimenting and run small examples as you learn.